Generative Adversarial Networks

Métodos Generativos, curso 2026-2027

Guillermo Iglesias, guillermo.iglesias@upm.es
Edgar Talavera Muñoz, e.talavera@upm.es

7 de noviembre de 2026

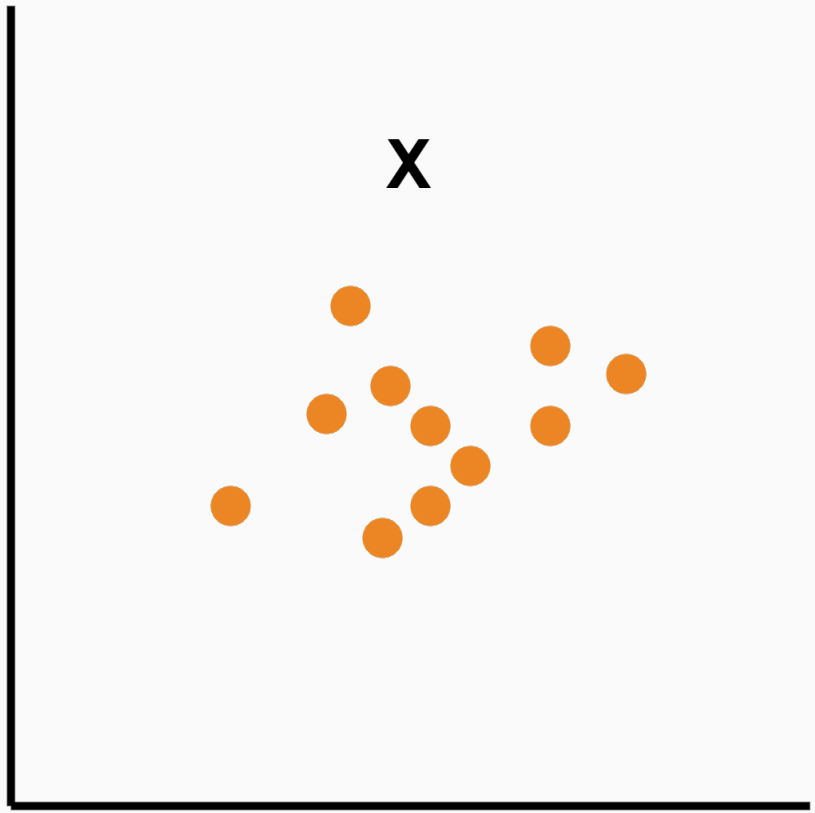
Escuela Técnica Superior de Ingeniería de Sistemas Informáticos | UPM



Modelos generativos

Modelos generativos

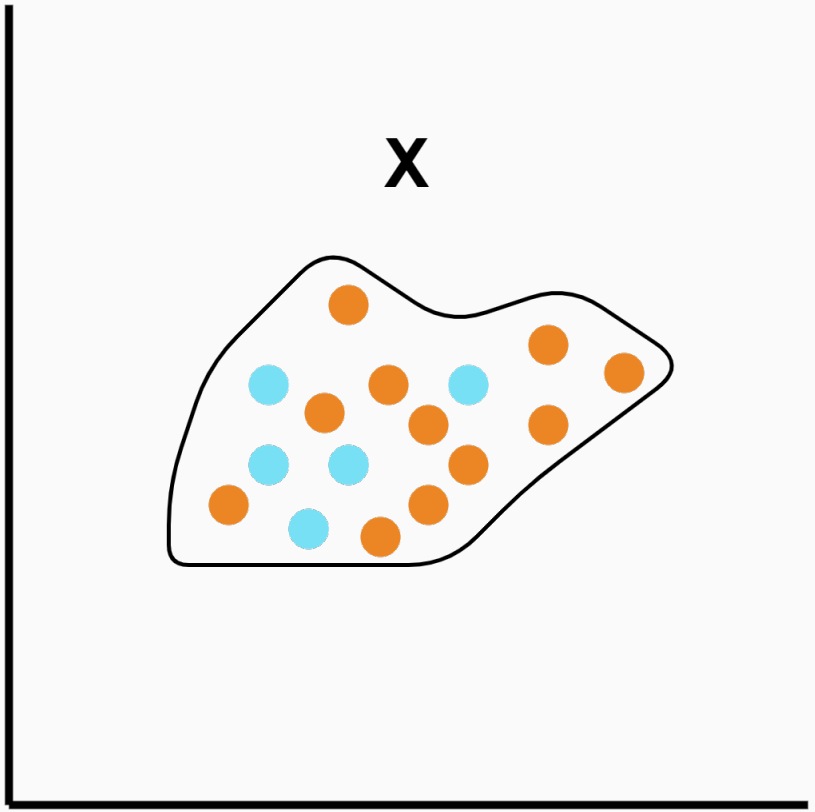
- Distribución de datos X de la cual se quieren generar nuevas instancias.



Distribución X.

Modelos generativos

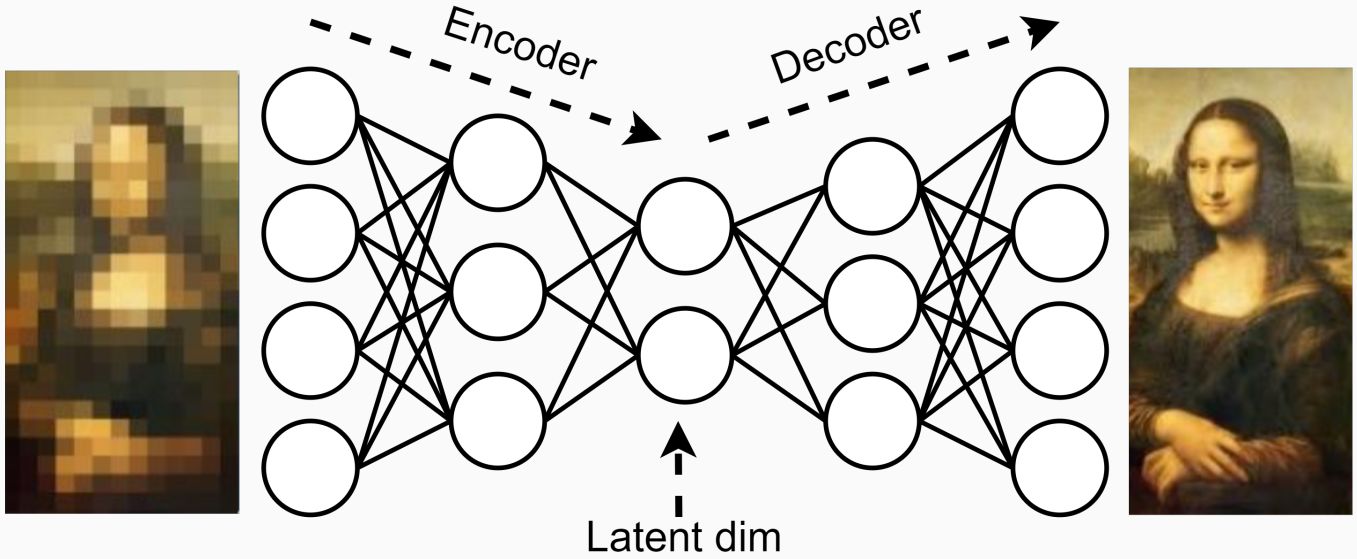
- Se quieren generar nuevos puntos distintos.
- Aprender la distribución de origen.
- Generar valores indistinguibles de los de entrada.



Nuevas instancias (en azul) de la distribución de datos X (en naranja).

Autoencoder

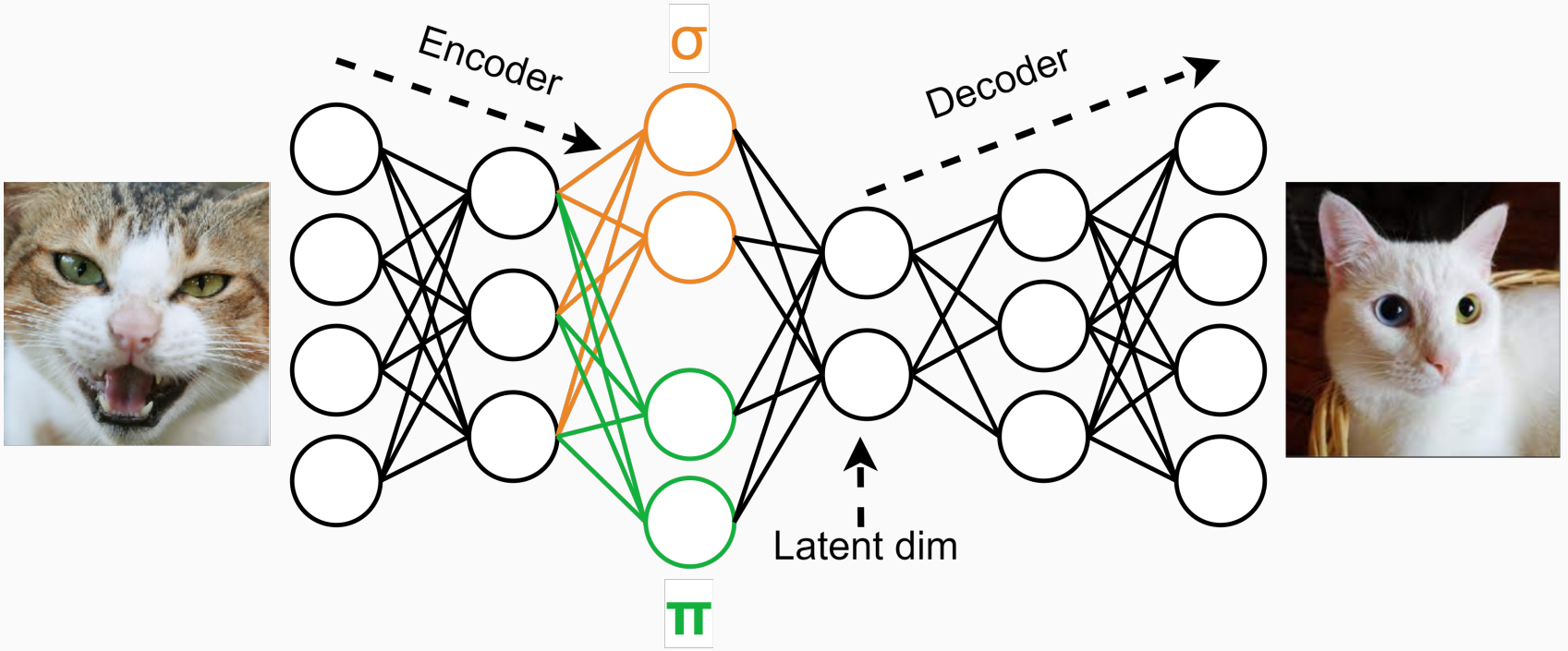
- Capaz de aprender la distribución de entrada.
- Sólo replica los datos de entrada.
- No consigue generalizar y generar nuevos datos.



Modelo de Autoencoder capaz de reconstruir una imagen de baja resolución.

Variational Autoencoder

- Aprende a generalizar gracias al uso de una función de densidad.
- Genera datos indistinguibles de los de entrada.
- Genera nuevos datos sin copiar directamente los ejemplos.



Modelo de Variational Autoencoder capaz de generar imágenes de gatos.

Modelos generativos: principales problemas

- Estos modelos no tienen la calidad suficiente.
- Se buscan alternativas capaces de generar resultados más nítidos.
- La aproximación de aprender directamente la distribución genera resultados poco diversos.

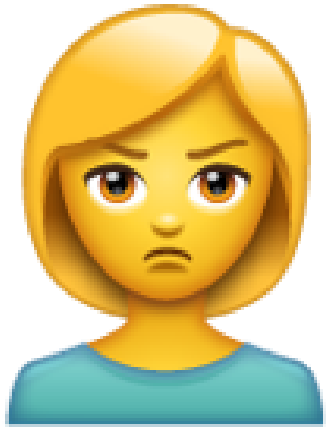
Generative Adversarial Network (GAN): Fundamentos

Definición

- Las redes GAN fueron propuestas en el año 2014 por Ian Goodfellow[1].
- Se definen como un modelo neuronal generativo basado en la teoría de juegos que tiene como objetivo replicar una distribución de datos.
- Se basa en un juego de suma cero que involucra 2 redes neuronales distintas.

Ejemplo: Falsificador vs. policía

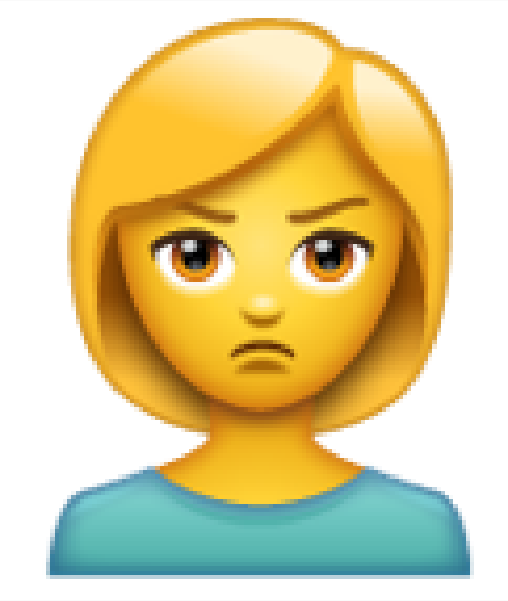
- Se quieren hacer falsificaciones de billetes.
- Se busca que la policía sea incapaz de distinguir entre los billetes falsos y los reales.





Redes GAN: Intuición previa

Falsificador



- Excelente artista.
- Es capaz de generar nuevos billetes.
- Inicialmente no sabe lo que es un billete.

Quiere engañar al policía con billetes falsos.

Policía



- Discrimina la veracidad de cada billete.
- Inicialmente no sabe lo que es un billete.

Quiere separar los billetes falsos de los verdaderos.

Redes GAN: Intuición previa

- El ladrón intenta engañar al policía generando billetes que se parezcan a los reales.
- Si engaña al policía sabe que ese billete es de buena calidad.



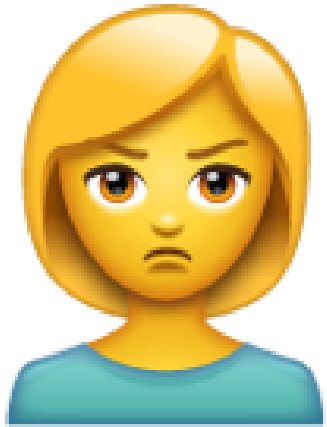
Redes GAN: Intuición previa

- El policía intenta no ser engañado discriminando las diferencias más sutiles entre los billetes.
- Si es engañado aprende cuáles son esos detalles que hacen que el billete sea falso.



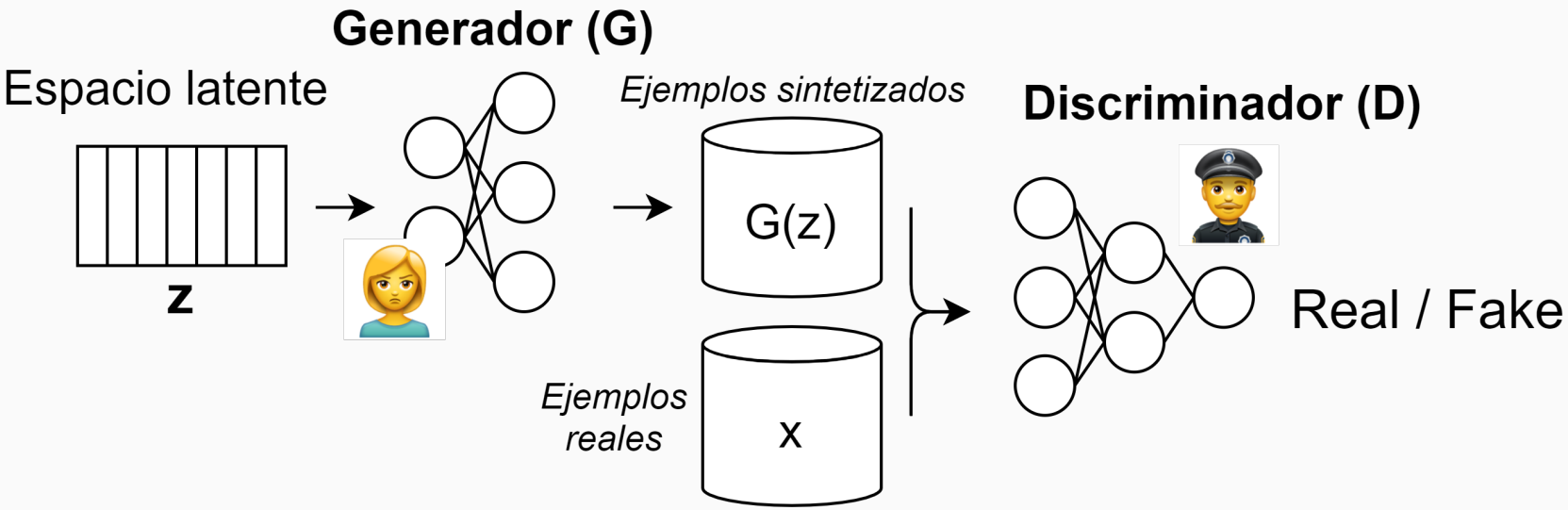
Redes GAN: Intuición previa

- El policía y el falsificador compiten por quien consigue vencer al otro.
- A medida que el tiempo pasa los billetes del falsificador son más realistas pero el policía aprende a diferenciar mejor los pequeños detalles.
- La competición entre ambos hacen que mejoren de manera simultánea y paralela.



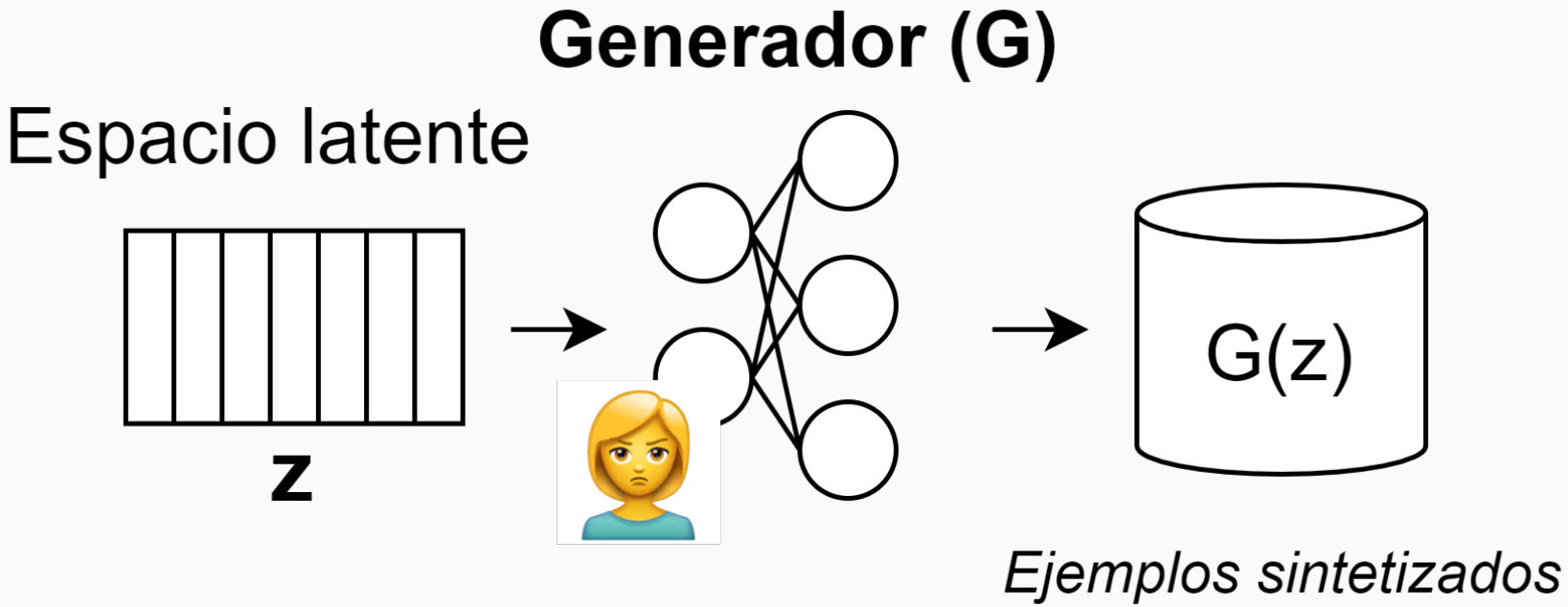


Esquema general de una red GAN



Arquitectura GAN.

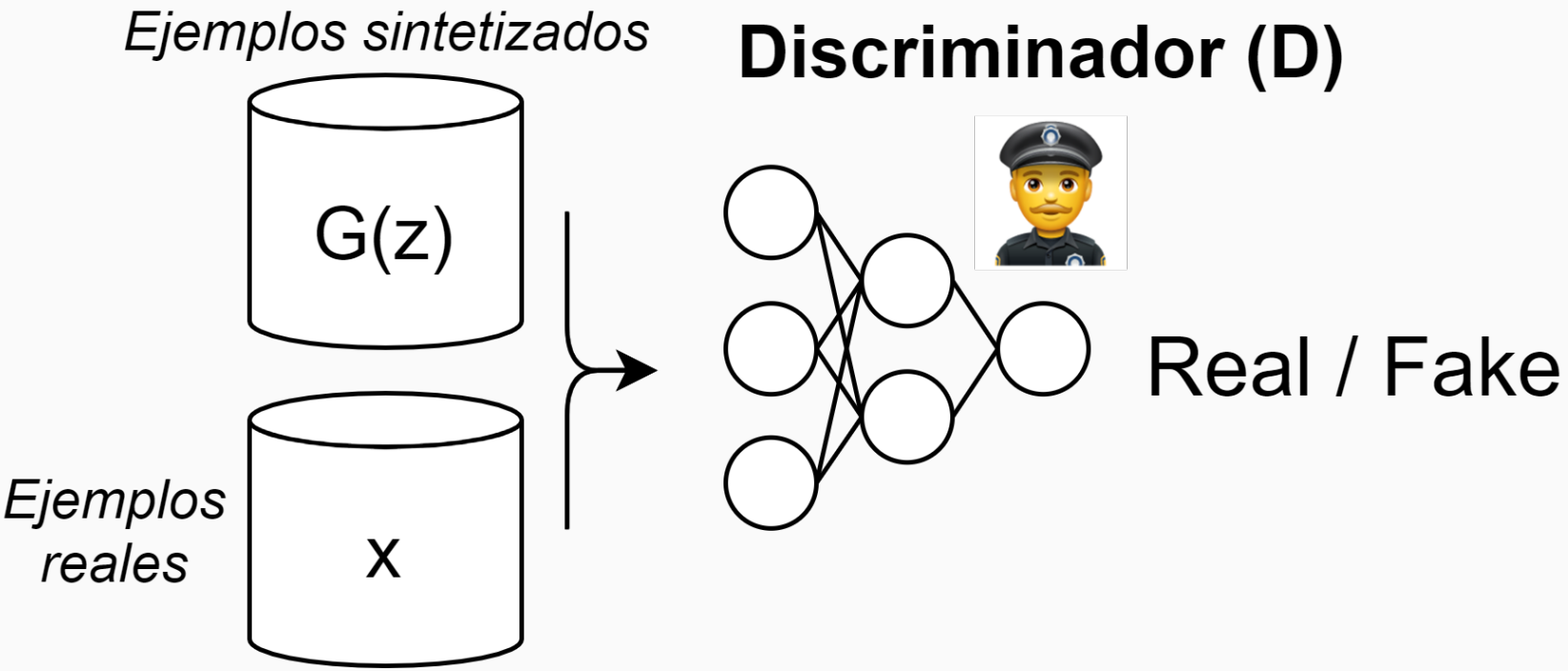
Generador



Arquitectura del generador.

Genera datos a partir de un espacio latente que le proporciona aleatoriedad.

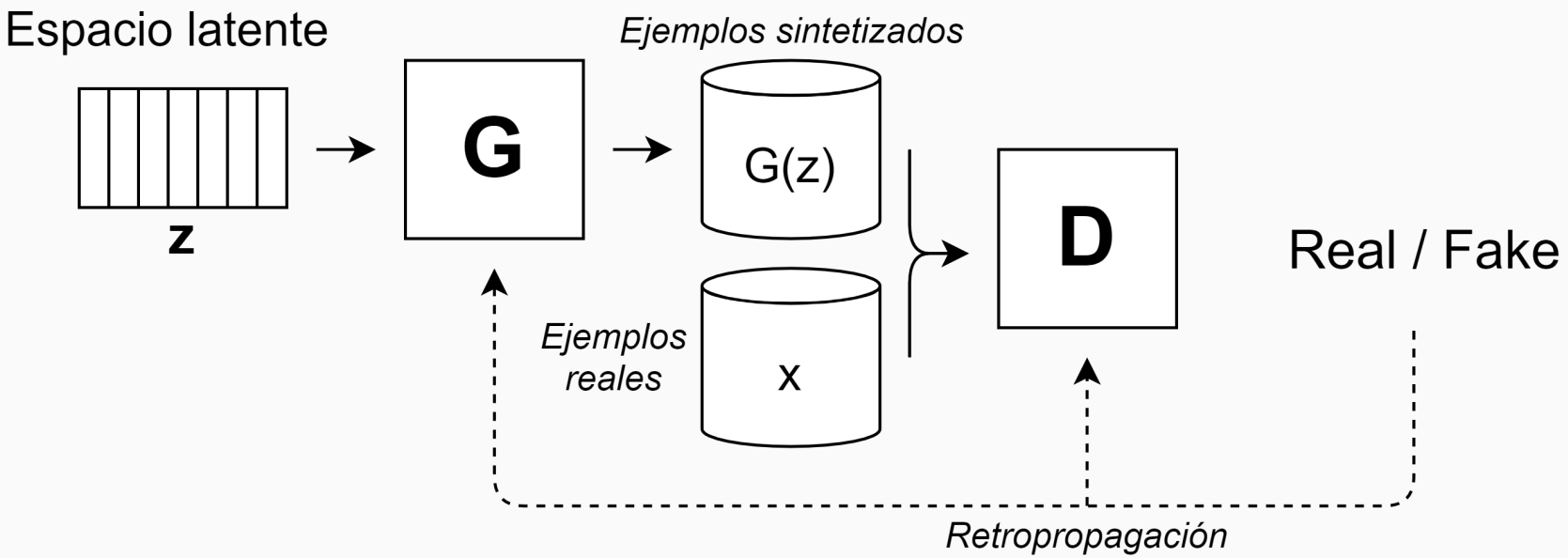
Discriminador



Arquitectura del discriminador.

Discrimina los datos a partir de la distribución real y la distribución generada por el generador.

Esquema general de una red GAN

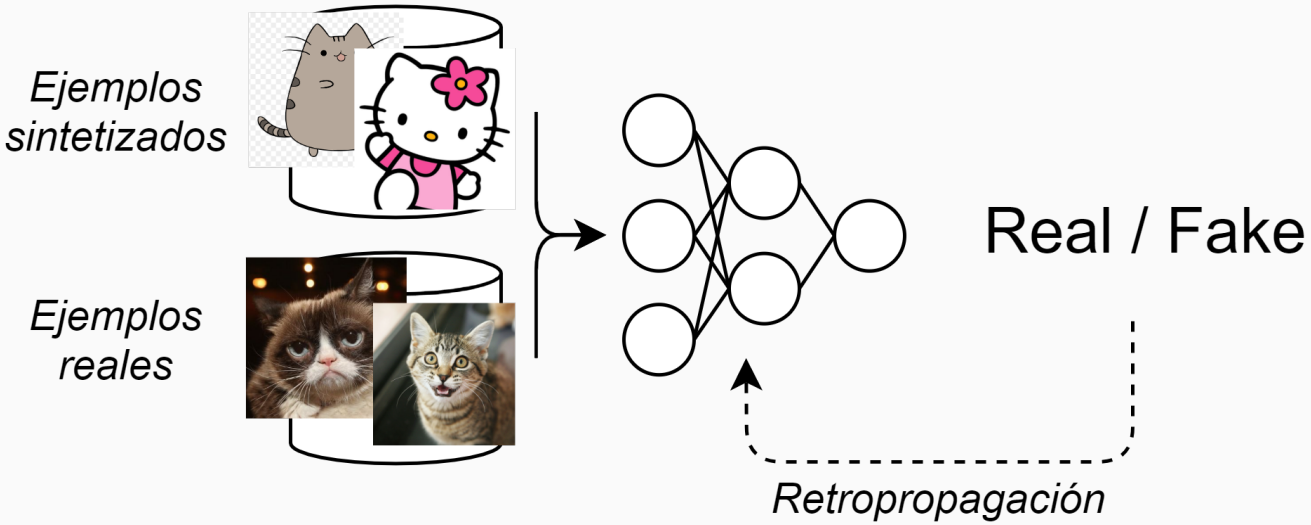


Arquitectura GAN.

- Durante el entrenamiento de una GAN se produce una competición entre G y D.
- En ella ambos modelos mejoran progresivamente de manera simultánea.

GAN: Entrenamiento

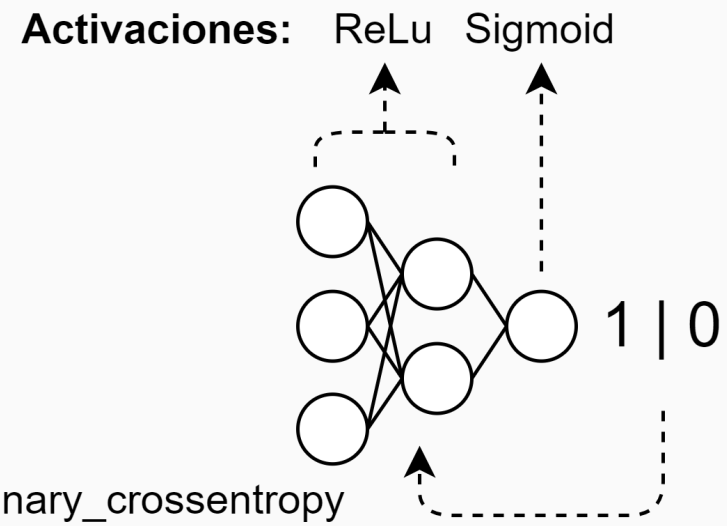
Entrenamiento del Discriminador



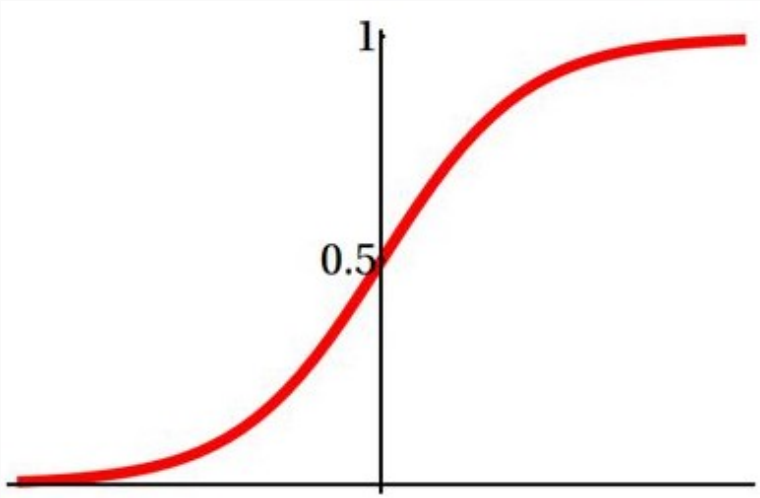
Entrenamiento de D.

- El D no es más que un clasificador binario.
- Su objetivo es diferenciar qué datos son reales y cuales falsos.
- Durante su entrenamiento realiza un aprendizaje para diferenciar ambas distribuciones.
- Se tiene que adaptar constantemente a las mejoras en las imágenes generadas por el G.

Entrenamiento del Discriminador



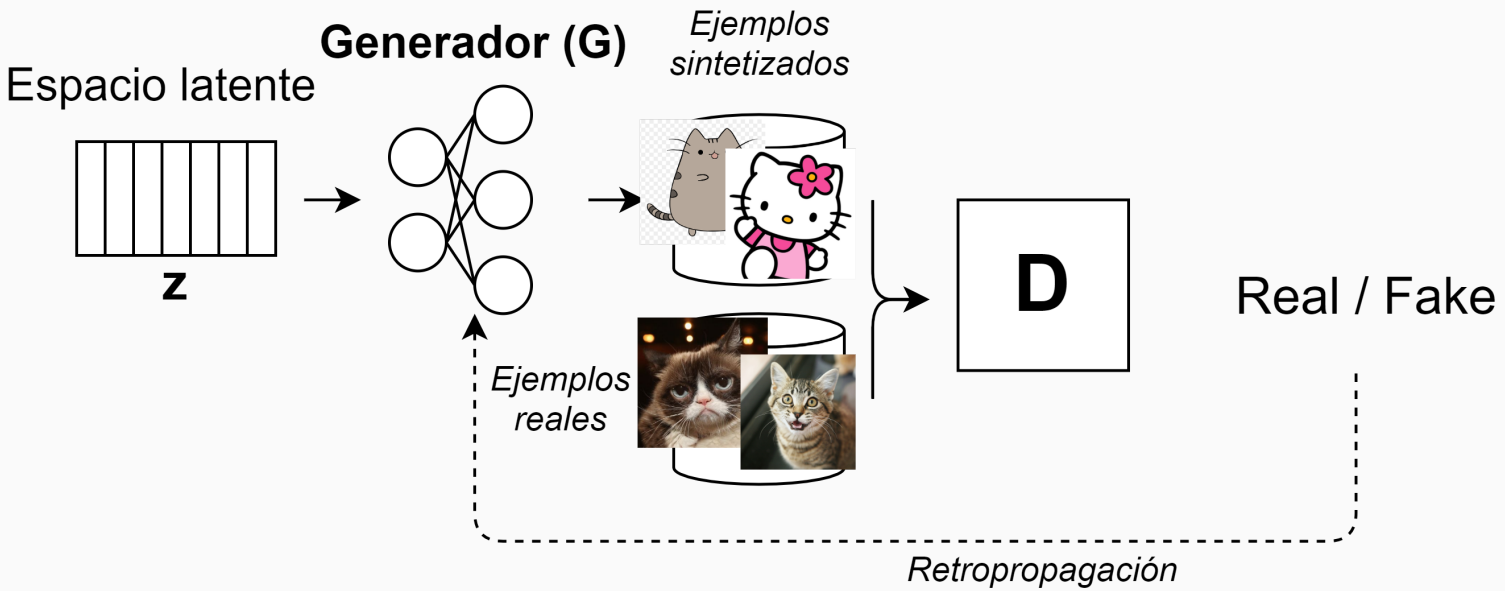
Pérdida y activaciones de D.



Función de activación sigmoide.

- Las activaciones de las capas ocultas y de entrada son cualquiera que permita actualizar los pesos correctamente.
- Al ser una clasificación binaria la activación de la capa de salida será la sigmoide (en el rango [0, 1]).
- La función de pérdida es la Entropía cruzada binaria.

Entrenamiento del Generador

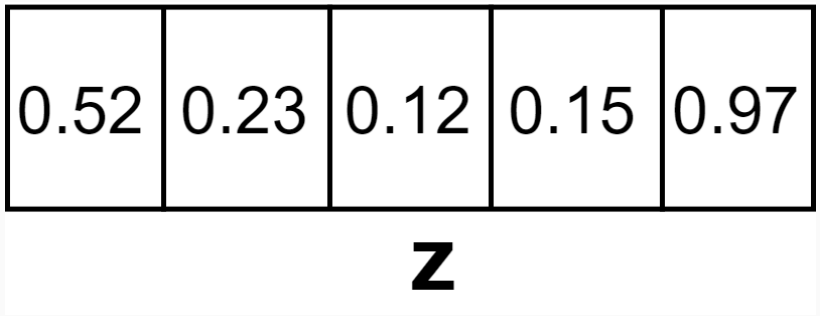


Entrenamiento de G.

- El G genera nuevos datos a partir del espacio latente.
- La actualización de pesos se hace a través del D.
- Su entrenamiento depende de si el D es capaz de diferenciar los datos.

Espacio latente

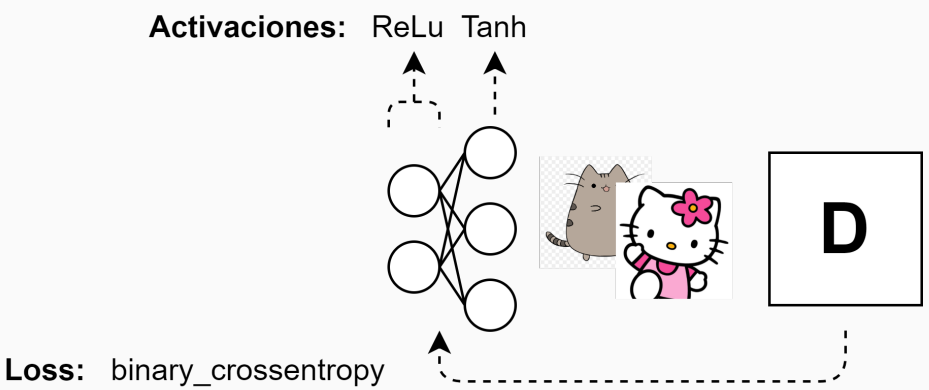
Se genera a través de ruido Gausiano o uniforme.



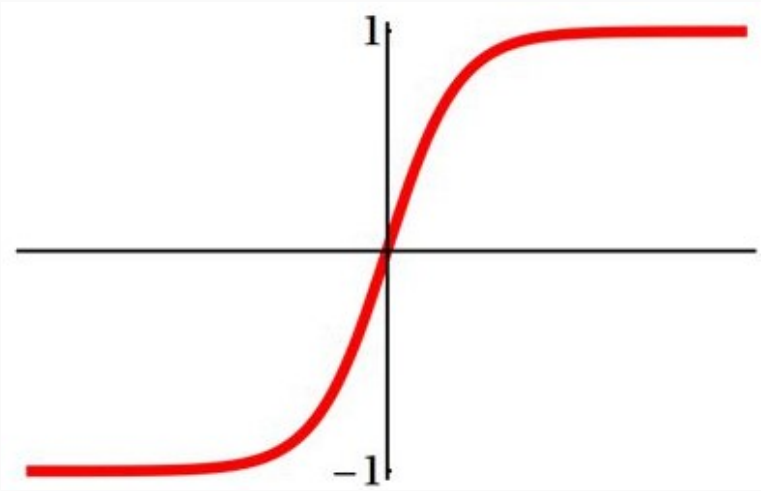
Vector de espacio latente.

- Es un vector n-dimensional de números reales.
- Actúa de semilla para la generación de nuevos datos.
- Introduce la aleatoriedad necesaria a la red.

Entrenamiento del Generador



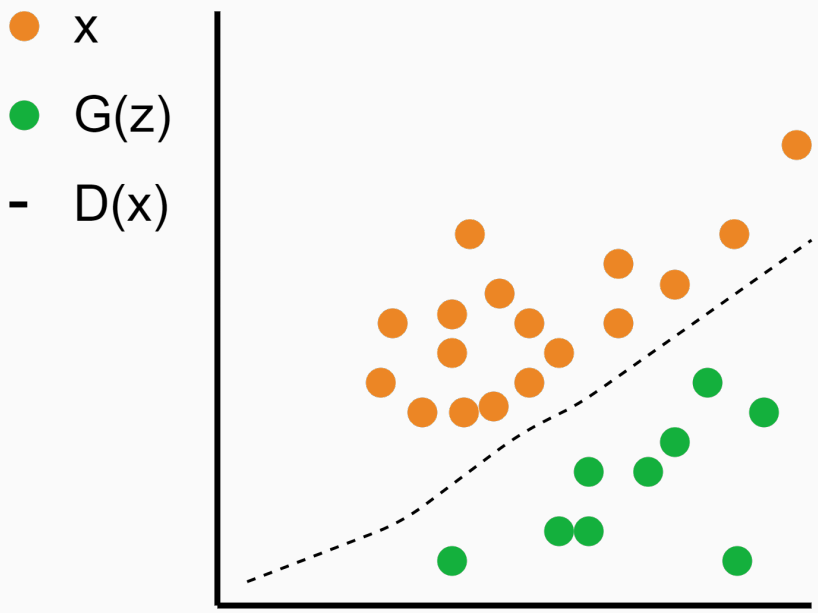
Pérdida y activaciones de G.



Función de activación tangencial.

- Las activaciones de las capas ocultas y de entrada son cualquiera que permita actualizar los pesos correctamente.
- La activación de la capa de salida será aquella que permita generar los datos de manera correcta.
- La función de pérdida es la Entropía cruzada binaria, con origen en la salida del D.

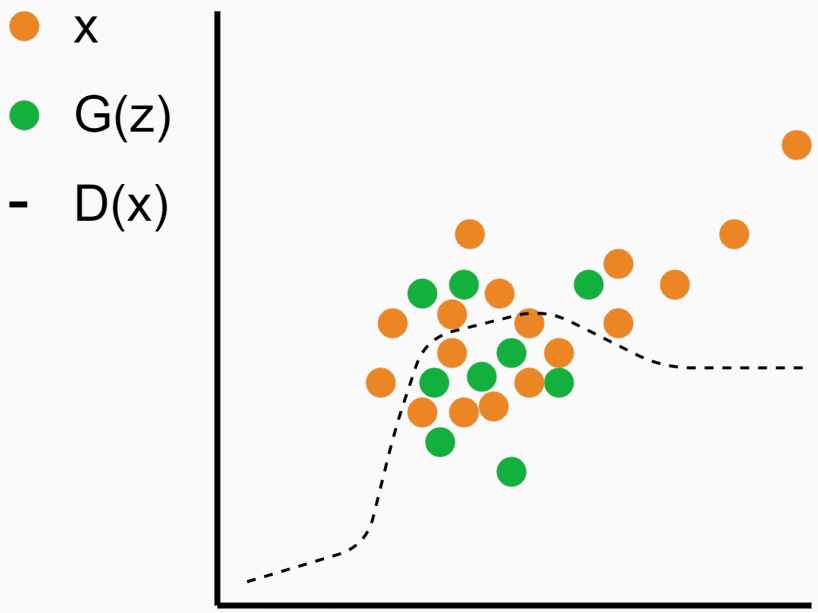
Entrenamiento: Primeras iteraciones



Distribución de datos reales (naranja) y generados (verde) durante las primeras iteraciones de un entrenamiento.

- Es muy fácil la diferenciación entre ambas distribuciones.
- Los datos generados son de muy mala calidad (ruido).

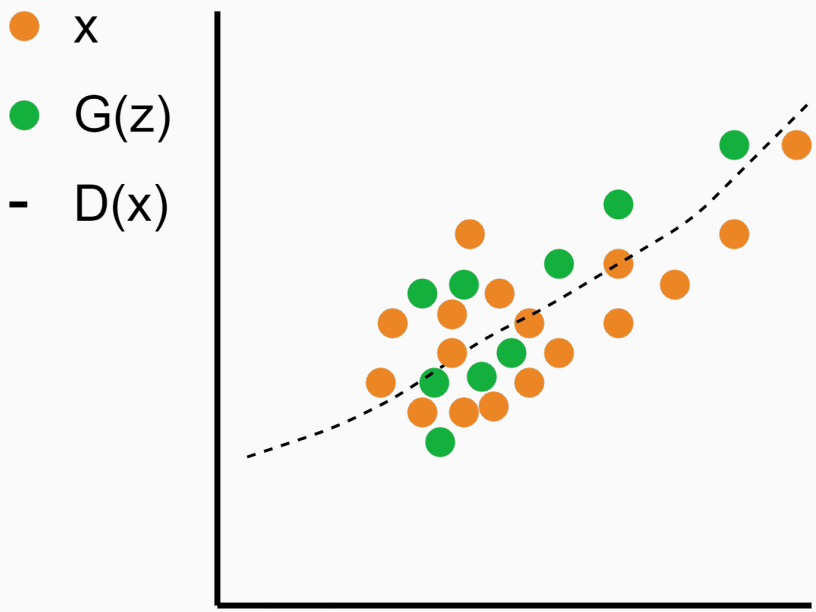
Entrenamiento: Iteraciones intermedias



Distribución de datos reales (naranja) y generados (verde) durante las iteraciones intermedias de un entrenamiento.

- El generador comienza a generar datos que engañan al discriminador.
- Todavía existen diferencias notables entre ambas distribuciones.

Entrenamiento: Final del entrenamiento



Distribución de datos reales (naranja) y generados (verde) tras haber finalizado un entrenamiento exitoso.

- Ambas distribuciones son indistinguibles.
- El discriminador es incapaz de diferenciar de qué distribución proceden los datos.
- Se ha llegado al equilibrio de Nash[2].

Minimax game

- El generador busca minimizar la siguiente expresión:

$$\log[1 - D(G(z))] \tag{1}$$

- El discriminador busca maximizar la siguiente expresión:

$$\log[D(x)] + \log[1 - D(G(z))] \tag{2}$$

Minimax game

- El juego de minimax entre ambos modelos se denota por la siguiente expresión, que corresponde con la divergencia de Jensen-Shannon:

$$\min_G \max_D L(D, G) = \log[D(x)] + \log[1 - D(G(x))] \quad (3)$$

- G intenta engañar a D.
- D intenta discriminar correctamente ambas distribuciones.

El Discriminador Óptimo

El Equilibrio de Nash se consigue cuando el entrenamiento está en equilibrio. Para entenderlo hay que entender cual es el mejor disciminador posible D_G^*.

Analizando la función objetivo (derivando respecto a $D(x)$ e igualando a cero), Goodfellow[1] demostró que el discriminador óptimo es:

$$D_G^*(x) = \frac{p_x(x)}{p_x(x) + p_g(x)} \tag{4}$$

Donde:

- $p_x(x)$: Distribución de probabilidad de los datos reales.
- $p_g(x)$: Distribución de probabilidad de los datos generados.

Fundamentación matemática: Equilibrio de Nash

En el Equilibrio de Nash, el generador es tan bueno que sus datos son absolutamente indistinguibles de los reales. Es decir, las distribuciones son exactamente iguales: $p_x(x) = p_g(x)$.

Si sustituimos esto en la ecuación del discriminador óptimo:

$$D_G^*(x) = \frac{p_x(x)}{p_x(x) + p_g(x)} = 0.5 \tag{5}$$

Es decir, el discriminador óptimo será incapaz de diferenciar los ejemplos reales y falsos[1].

[1] De ahí el 0.5.

Conexión con la Divergencia Jensen-Shannon

Sustituyendo el discriminador óptimo D_G^* en la funcion Minimax original[2], se obtiene:

$$\min_G V(G, D_G^*) = -\log(4) + 2 \cdot JSD(p_x \parallel p_g) \tag{6}$$

- La Divergencia Jensen-Shannon (JSD) mide la distancia entre dos distribuciones de probabilidad.
- Dado que la JSD siempre es ≥ 0, el mínimo global de esta función se alcanza única y exclusivamente cuando $JSD = 0$.
- Esto demuestra matemáticamente que optimizar el juego Minimax equivale a forzar que la distribución generada encaje perfectamente en la real.

[2] No hace falta entrar en cómo se llega a esta ecuación, eso se lo dejamos a los matemáticos.

GAN: Problemas principales

Problemas de las GAN

Las GAN cuentan con una serie de problemas comunes. Estos suelen presentarse en los entrenamientos de una GAN debido a las particularidades de su arquitectura.

Se revisarán los siguientes:

- Inestabilidad
- Problemas derivados del gradiente:
  - Gradient explosion
  - Gradient vanishing
- Mode Collapse

Inestabilidad

El problema de la inestabilidad se debe a que las GAN cuentan con 2 redes neuronales al mismo tiempo.

- Las redes neuronales de por sí son modelos inestables.
- Al hacer que dos modelos interactúen entre sí se aumenta esta inestabilidad.
- Mantener una coordinación entre ambas redes es un proceso delicado.

Inestabilidad

Para intentar evitar la inestabilidad se suelen tomar una serie de precauciones que intenten equilibrar el entrenamiento:

- Que las arquitecturas de la D y la G tengan la misma forma pero inversa.
- En caso de que el D sea muy certero (caso más habitual), limitar su potencia haciendo uso de regularización.

Que las dos redes no estén en sincronía puede hacer que ninguna entrene correctamente, ya que el G necesita que el D falle de vez en cuando y viceversa.

Problemas del gradiente

Los problemas derivados del gradiente son comunes a todas las redes neuronales. Estos están directamente influenciados por el número de capas de la red.

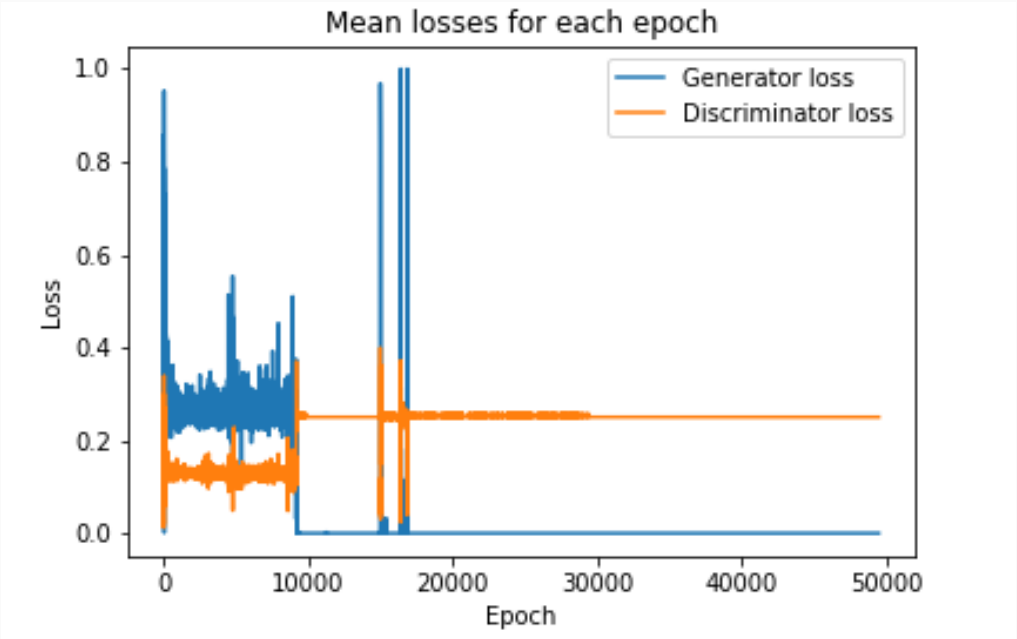
Se diferencian dos tipos:

- Gradient explosion.
- Gradient vanishing.

Al realizarse la retropropagación los valores de pérdida pasan de unas capas a otras. En este algoritmo las derivadas de cada neurona pueden llegar a descontrolarse.

$$W'_x = W_x - \alpha \left(\frac{\partial \text{Loss}}{\partial W_x} \right) \tag{7}$$

Problemas del gradiente: Gradient explosion

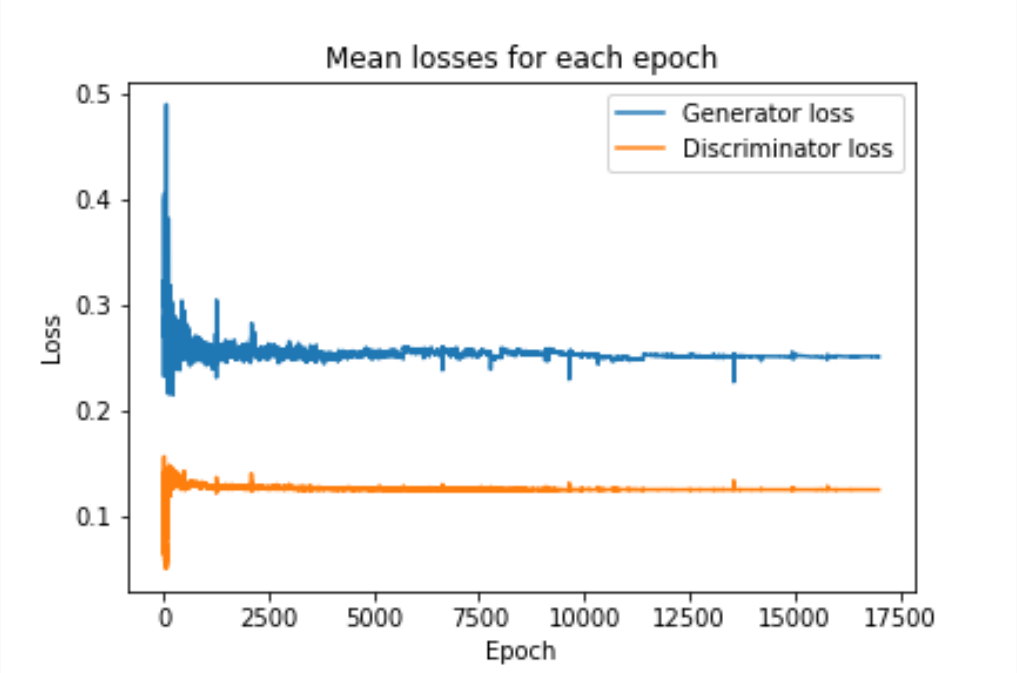


Gradient explosion.

El gradient explosion, también conocido como exploding gradients sucede cuando la actualización de pesos toma valores muy elevados.

Se identifica con valores de pérdidas de NaN o muy exageradas.

Problemas del gradiente: Gradient Vanishing

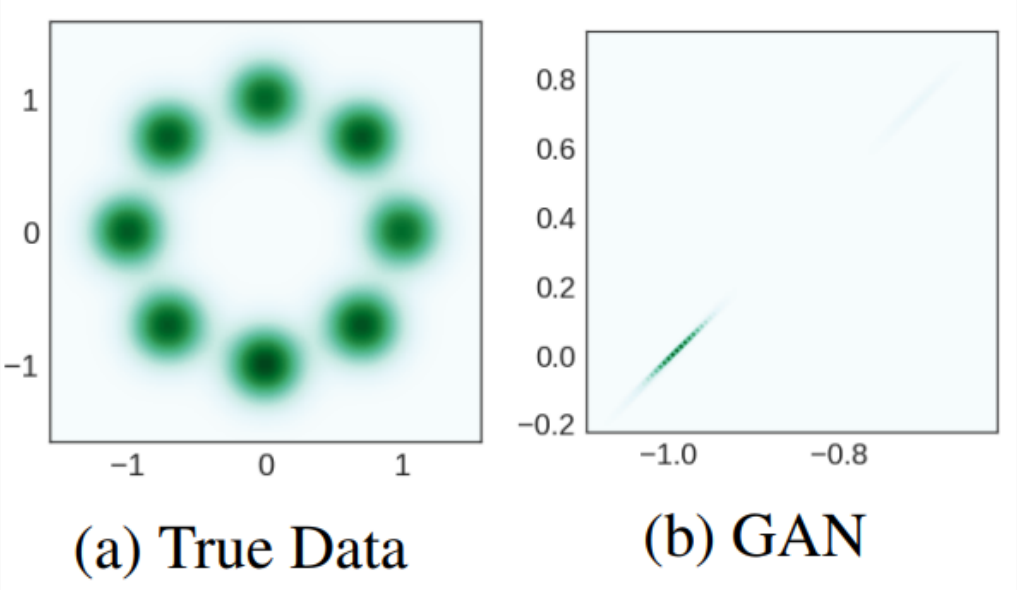


Gradient vanishing.

Cuando sucede gradient vanishing, también llamado vanishing gradients, la actualización de pesos se hace nula por tener valores muy pequeños.

Se identifica cuando la pérdida es constante en el tiempo.

Mode Collapse



[3]

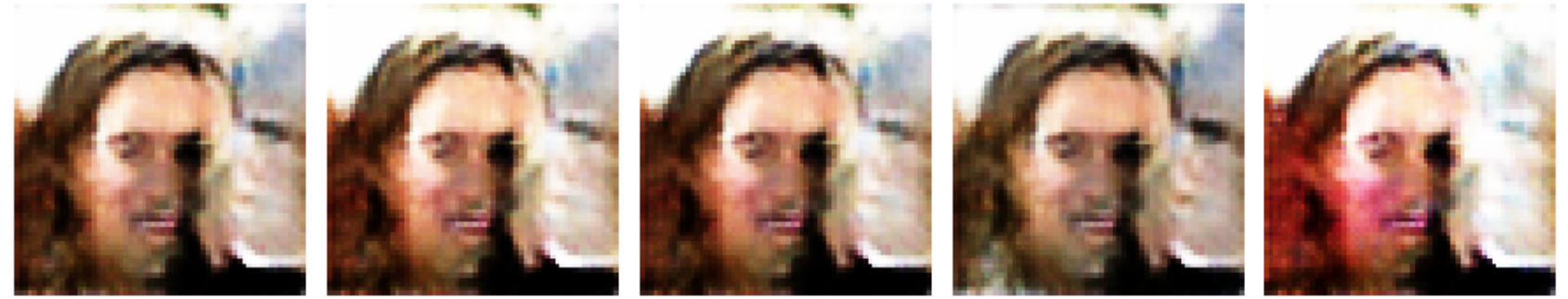
El colapso modal sucede cuando la información generada sólo pertenece a un subgrupo de la total.

Mode Collapse: Definición profunda

El colapso modal sucede cuando la información generada sólo pertenece a un subgrupo de la total [3].

A nivel práctico, esto significa que el Generador ha encontrado una (o unas pocas) muestras que consiguen engañar siempre al Discriminador actual.

El Generador decide ignorar el vector de ruido de entrada z y mapea todo el espacio latente hacia esa misma salida.



El modelo es incapaz de producir diversidad [4].

La raíz matemática del Mode Collapse

El problema surge de la dinámica del juego Minimax:

$$\min_G \max_D V(G, D) \tag{8}$$

En la práctica, suele suceder que G entrena demasiado rápido y gana ventaja sobre D.

El generador converge hacia un único punto x^* que el discriminador actual (imperfecto) clasifica como real, colapsando la función de manera que $G(z) \to x^* \ \forall z$[3].

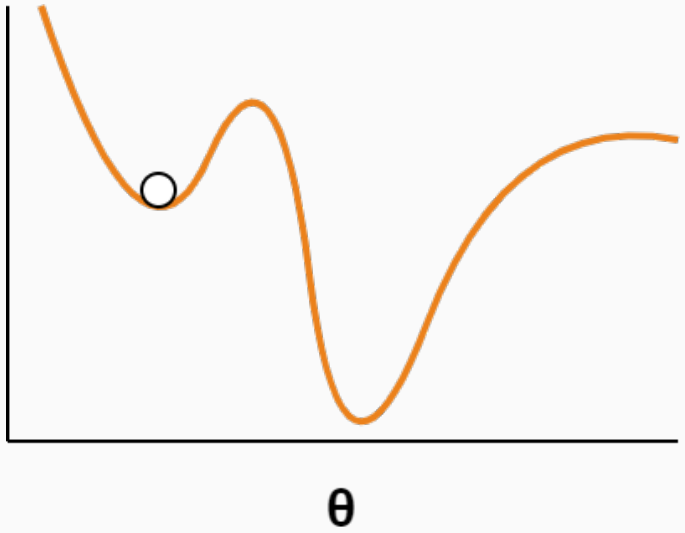
[3] Es decir, siempre genera el mismo ejemplo x^*.

Mode-Seeking

Las GANs tradicionales sufren de un comportamiento conocido como Mode-Seeking (búsqueda de modos).

En estadística, una distribución multimodal tiene varios ”picos” (modos).

- La red prefiere generar un solo tipo de dato perfecto (cubrir un único modo) antes que intentar abarcar toda la diversidad de los datos y arriesgarse a generar muestras irreales (quedarse en los valles).



Minibatch Discrimination

Una de las primeras soluciones [5] fue modificar el comportamiento del Discriminador. En lugar de evaluar imágenes de una en una, el Discriminador evalúa un batch entero[4].

Además, se añade una capa que calcula la distancia o similitud entre todas las imágenes del batch. Si el Generador envía 32 imágenes idénticas, el Discriminador detecta que la varianza del lote es cero y las clasifica como falsas inmediatamente, forzando a G a diversificar.

[4] Que, por otra parte, es mucho más eficiente.

Wasserstein GAN (WGAN)

La solución teórica y matemática definitiva al Mode Collapse llegó con la Wasserstein GAN (WGAN) [6].

Se utiliza la pérdida de Wasserstein (Earth Mover's Distance), que mide el "coste" de transportar masa para convertir una distribución en otra en vez de la divergencia de Jensen-Sharon.

$$W(p_{data}, p_g) = \inf_{\gamma \sim \Pi(p_{data}, p_g)} \mathbb{E}_{(x,y)\sim\gamma}[||x - y||] \tag{9}$$

Esta distancia proporciona gradientes suaves y continuos en todo el espacio. El Generador siempre recibe una señal útil sobre hacia dónde moverse, evitando por completo que se quede atascado en un solo modo.

Métricas de evaluación de métodos generativos

La ”calidad” de los datos generados

¿Cómo sabemos si una GAN genera “buenas” imágenes?

A diferencia de modelos discriminativos (con métricas objetivas como accuracy o loss), los modelos generativos no tienen una métrica de evaluación directa.

La función de pérdida no se correlaciona bien con la calidad percibida.



Dos imágenes pueden ser igualmente *probables* pero muy diferentes visualmente [3].

Métricas tradicionales

- Mean Squared Error (MSE) o derivados loss no reflejan la percepción humana ni la diversidad del conjunto.

Necesitamos métricas que capturen al mismo tiempo:

- Realismo: Que los datos generados sean similares a los del *dataset* original.
- Diversidad: Que los datos generados no sean siempre iguales.
- Originalidad: Que los datos generados no sean copias del dataset.

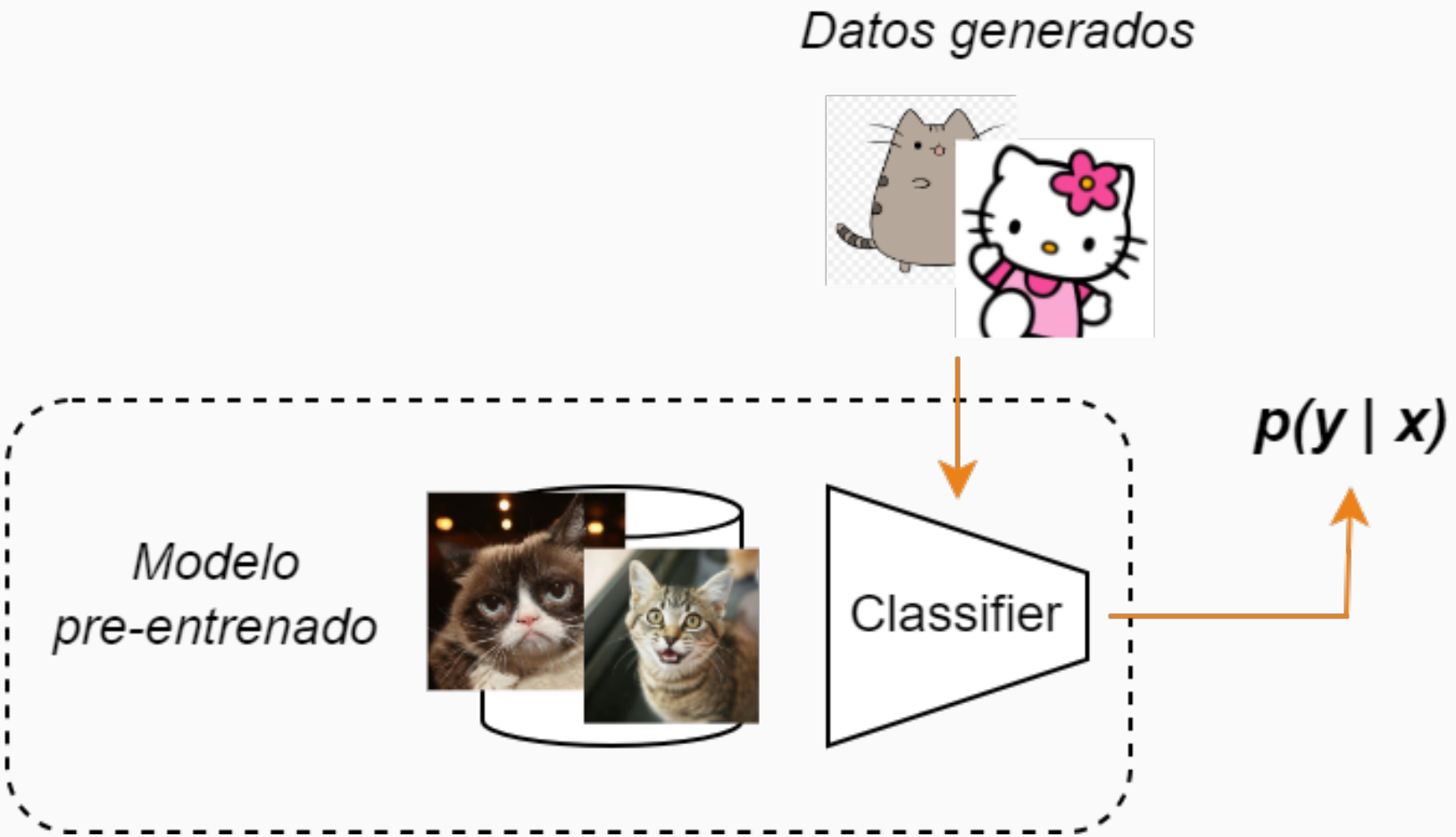
Usando un modelo auxiliar

La principal solución es utilizar un modelo externo pre-entrenado que evalue la calidad de los datos generados.

De esta manera se analizará el comportamiento del modelo ante datos generados.

- Se busca que el comportamiento del dataset generado sea realista
- El modelo pre-entrenado no entrena ni sabe nada de los datos generados

Usando un modelo auxiliar



normalmente se usa como modelo pre-entrenado la red* ***Inceptionv3* *[7] entrenada con el dataset Imagenet*

Inception Score (IS) [5]

- **Definición**

$$IS = exp(E_x[KL(p(yx)p(y))]) \tag{10}$$

- $p(y \mid x)$: Predicciones de la red para los datos generados
- $p(y)$: Distribución de etiquetas predichas por el modelo

Se busca que las predicciones sean de una clase concreta. Lo que significaría que la imagen es realista.

Características de la IS [5]

- Fácil de calcular
- No mide similitud con los datos reales, sólo tiene en cuenta los generados
- Depende del clasificador
  - Funciona mejor cuanto más similar sea el dataset a imagenet
    - Que es un dataset de fotos reales

Fréchet Inception Distance (FID) [8]

- Definición

$$FID = \mu_r \mu_g{}^2 + Tr(\Sigma_r + \Sigma_g 2(\Sigma_r \Sigma_g)^{1/2}) \tag{11}$$

- Compara las distribuciones (media μ y covarianza Σ) de features de Inception entre datos reales y generados
- Las features son las salidas de una de las últimas capas de InceptionV3, normalmente la penúltima

Características de la FID [8]

- Compara la distribución de datos real y la generada
- Mide realismo y diversidad
- Sensible al tamaño de la muestra
- Supone que las predicciones se organizan con una distribución gausiana

Kernel Inception Distance (KID) [9]

- Definición

$$KID = MMD^2(\phi(x_r), \phi(x_g)) \tag{12}$$

- Calcula la Maximum Mean Discrepancy (MMD) entre las features de Inceptionv3
- De esta manera no se asume nada sobre las distribuciones de las features
- Es especuialmente útil para muestras pequeñas

Aplicaciones de las GAN

Data augmentation con GAN

Data Augmentation consiste en generar nuevos datos de un dataset para aumentar el número de ejemplos. Esto es especialmente útil cuando se cuenta con conjuntos de datos insuficientes para entrenar, por ejemplo, una red neuronal.

También toma especial importancia cuando se cuenta con desbalanceo entre clases de un dataset.

Las GAN tienen como principal ventaja que generan nuevos datos muy similares a la distribución de datos original, pero sin copiar ni modificarlos directamente.

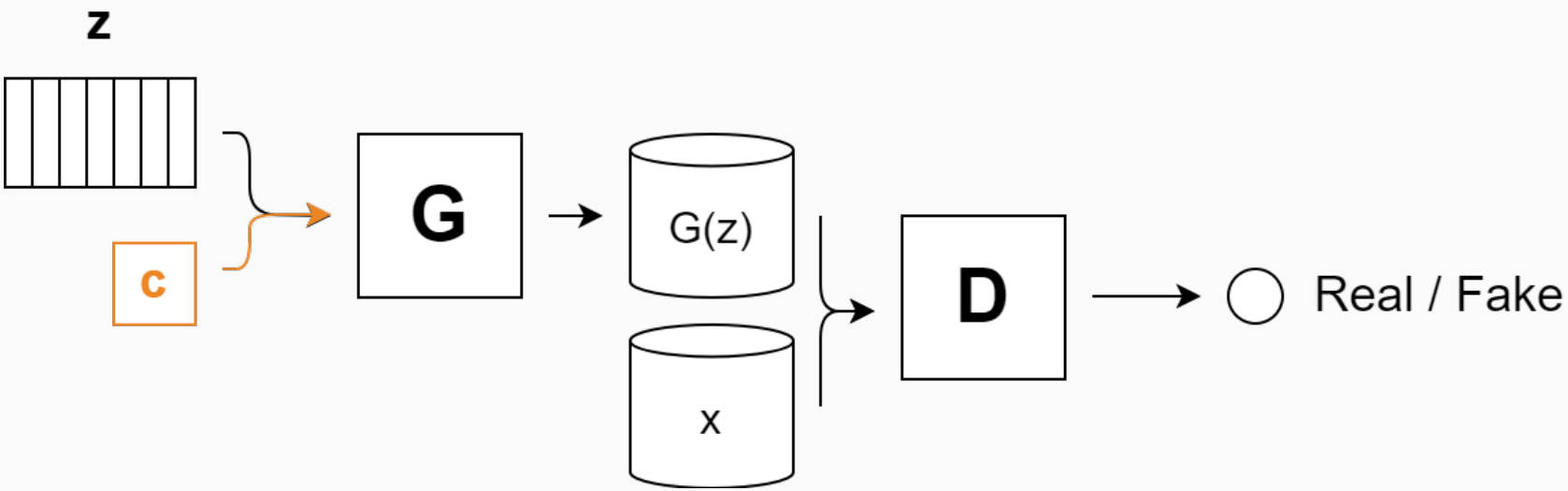
Generación Condicionada (CGAN)

En una GAN clásica, el generador toma un vector de ruido aleatorio z y produce una imagen. En este caso, no se tiene ningún control sobre qué tipo de imagen se va a generar

Para solucionar esto, surgieron las Conditional GANs (CGAN) [10].

- Se introduce una variable condicional c (como una etiqueta de clase, texto, o incluso otra imagen).
- Esta condición c se inyecta tanto en el Generador (junto con el ruido z) para guiar la generación, como en el Discriminador (junto con la imagen x).
- Esto permite dirigir qué características concretas tendrá la salida generada: $G(z, c)$.

Conditional GAN (CGAN)



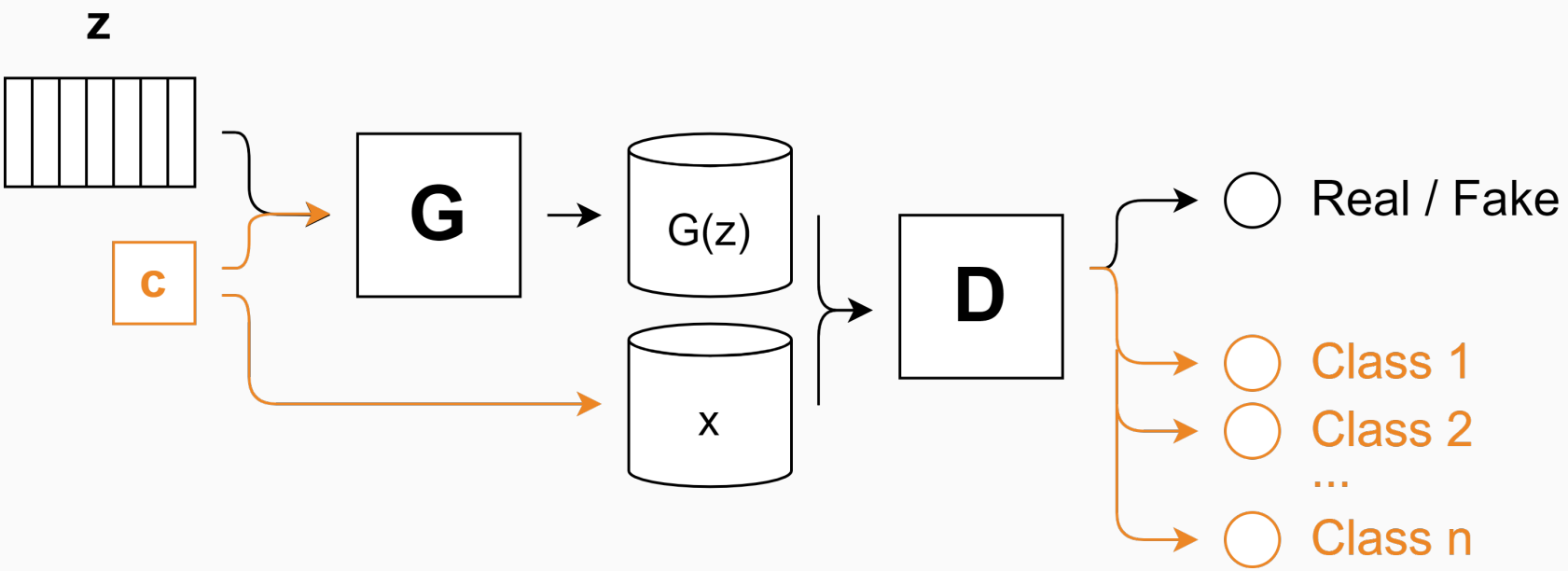
Arquitectura de una CGAN

ACGAN: Auxiliary Classifier GAN [11]

La arquitectura Auxiliary Classifier GAN (ACGAN) lleva el concepto del condicionamiento un paso más allá para mejorar la calidad de las imágenes. En lugar de recibir la clase como entrada, el discriminador se encarga de predecirla.

- Es capaz de separar en distintas clases los datos con los que trata.
- Realiza un entrenamiento supervisado usando las etiquetas del dataset.
- El discriminador distingue además de la veracidad de cada dato, la clase a la que pertenece.
- De esta manera es capaz de aprender los distintos tipos de datos.

ACGAN: Auxiliary Classifier GAN [11]



Arquitectura de una ACGAN

ACGAN: Función de Pérdida

Dado que el discriminador de la ACGAN ahora tiene dos tareas simultáneas, su función de pérdida se divide en dos componentes independientes:

- L_S: discriminar si la imagen es real o generada.

$$L_s(D, G) = \log[D(x)] + \log[1 - D(G(x))] \tag{13}$$

- L_C: clasificar el tipo de imagen.

$$\min_G \max_D L_c(D, G) = \log P(C = c|x) + \log P(C = c|G(z)) \tag{14}$$

$$L_C = E[\log P(C = c|X_{real})] + E[\log P(C = c|X_{fake})]$$

El Discriminador maximiza $L_S + L_C$, mientras que el Generador busca maximizar la clasificación correcta pero engañar en la fuente, maximizando $L_C - L_S$.

ACGAN[11]

- Capaz de separar en distintas clases los datos con los que trata.
- Realiza un entrenamiento supervisado usando las etiquetas del dataset.
- El discriminador distingue además de la veracidad de cada dato, la clase a la que pertenece. De esta manera es capaz de aprender los distintos tipos de datos.



Resultados de una ACGAN para la generación de dígitos del dataset MNIST[12]

Domain-to-domain translation

El Domain-to-domain translation es una aplicación no exclusiva de las GAN. Sin embargo estas redes funcionan muy bien para esta aplicación.

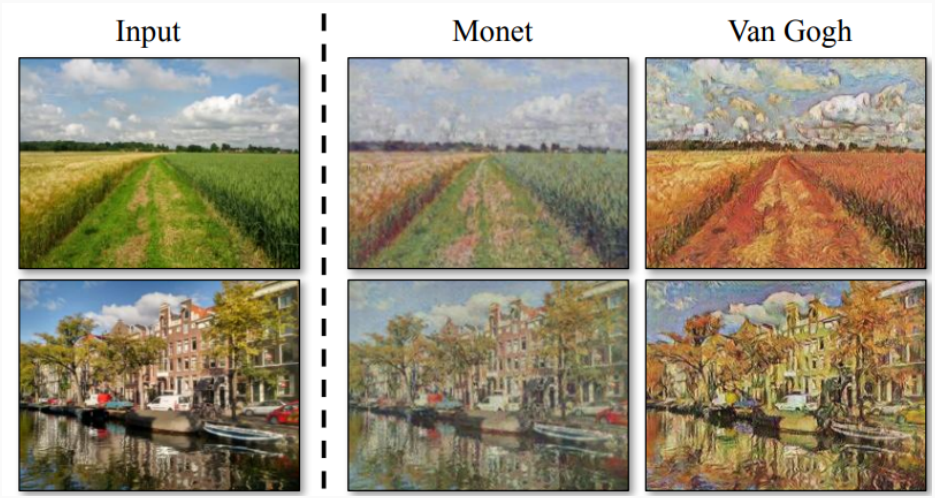
Es una de las aplicaciones más vistosas y que más ha ayudado a la popularidad de las GAN en os últimos años.

- La idea es traducir un conjunto de datos de entrada de un dominio a otro distinto.

Domain-to-domain translation: Ejemplos



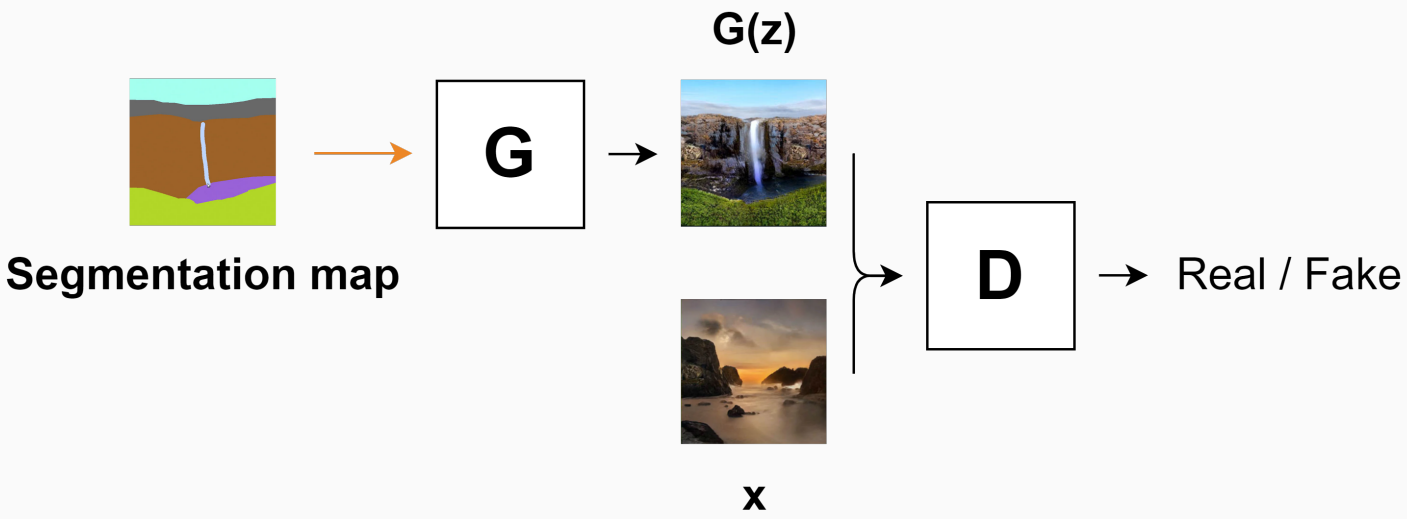
Resultados de la arquitectura TAC-GAN[13].



Resultados de la arquitectura CycleGAN[14].

Image-to-Image Translation with Conditional Adversarial Nets (pix2pix)[15]

- La arquitectura pix2pix fue presentada para realizar generación de imágenes condicionada por una entrada, que también es una imagen.
- La idea es traducir la imagen de entrada a otro dominio, en lo que se conoce como image-to-image translation.

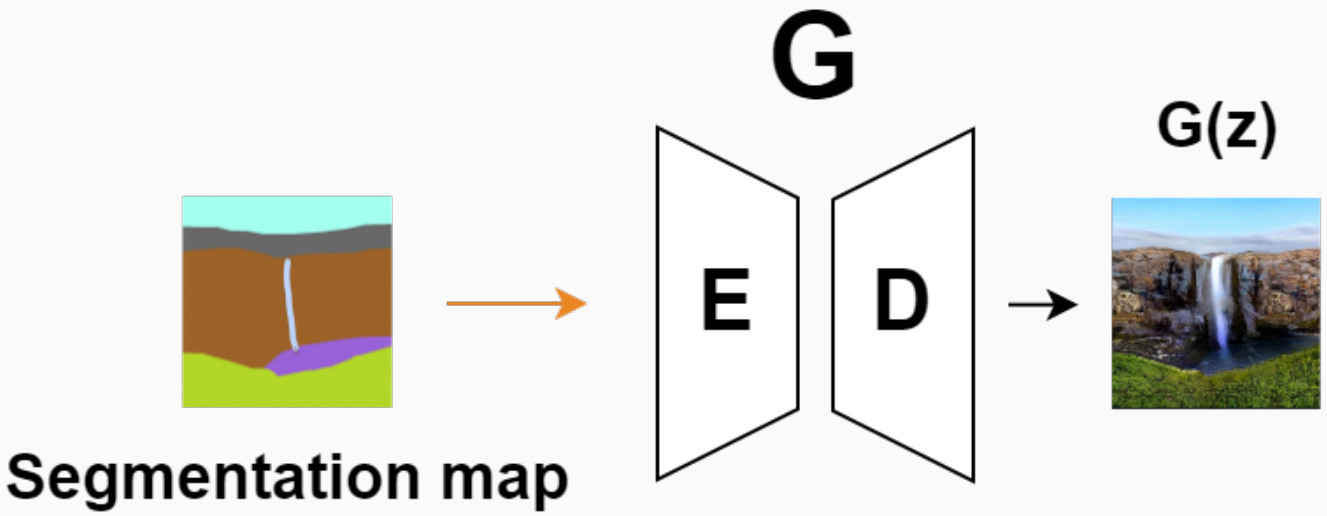


Arquitectura general de una pix2pix.

pix2pix[15]: Traducción de imagen a imagen

A diferencia de las GAN clásicas que parten de un vector de ruido z, en pix2pix la entrada principal es una imagen condicionante x.

Esto cambia radicalmente el enfoque del Generador, que pasa a ser un Autoencoder y no necesitar un espacio latente.



pix2pix[15]: Función de Pérdida combinada

La pérdida adversaria se combina para equilibrar el realismo y la fidelidad:

- Pérdida Condicional (cGAN): El discriminador evalúa pares de imágenes; la original condicionante y la real o generada.

$$\min_G \max_D \mathcal{L}_{cGAN}(G, D) = \log D(x, y) + \log(1 - D(x, G(x, z))) \tag{15}$$

  donde x es la imagen de entrada e y y $G(x, z)$ las del nuevo dominio.

- Pérdida L1: Para evitar que el generador "alucine" artefactos[5], se usa el Mean Absolute Error (MAE).

$$\min_G \mathcal{L}_{L1}(G) = \mathbb{E}_{x,y,z}[||y - G(x, z)||_1] \tag{16}$$

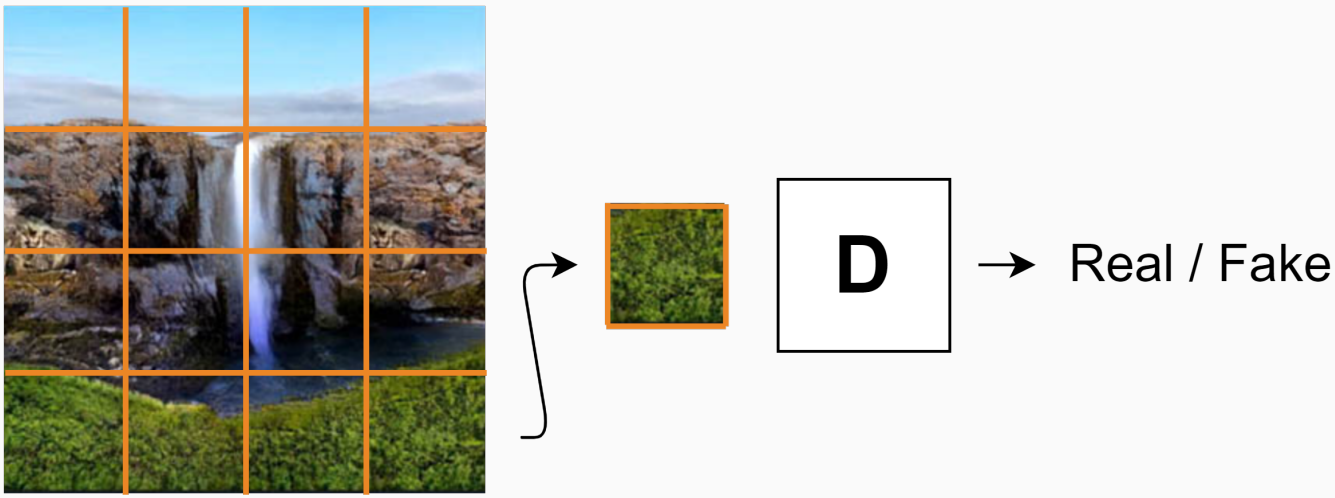
[5] Sí, las alucinaciones no son exclusivas de los transformers.

pix2pix: El Discriminador PatchGAN

La pérdida L1 captura la estructura global pero genera imágenes borrosas.

Para solucionarlo, pix2pix introduce la arquitectura PatchGAN [15]:

- En lugar de clasificar la imagen completa con un único valor (Real/Falso), el discriminador evalúa la imagen en parches locales de $N \times N$ píxeles (comúnmente 70×70).
- De esta manera se consigue que cada sección de la imagen sea realista.



Discriminador de la arquitectura PatchGAN, basado en [15].

El problema de los datos no emparejados

La pix2pix requiere datos estrictamente emparejados para calcular la pérdida L1 (e.g., la foto exacta del mismo edificio y su boceto).

Sin embargo, para muchos problemas, obtener este tipo de conjuntos de datos es imposible.

- ¿Cómo traducimos una pintura de Monet a una foto real? No existe la foto original que Monet miró.
- ¿Cómo traducimos caballos a cebras? No tenemos fotos del mismo animal en la misma pose exacta como caballo y como cebra.

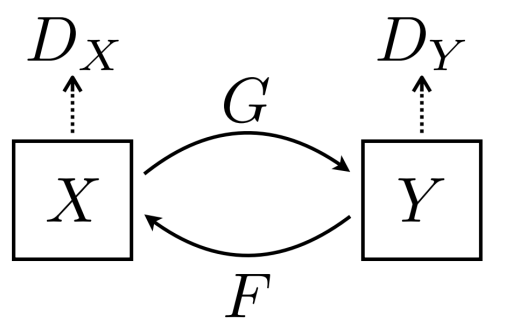
CycleGAN [14] se presentó en 2017 para resolver el problema de la traducción Unpaired Domain-to-Domain.

CycleGAN[14]

CycleGAN asume que si traducimos del dominio X al dominio Y, y luego traducimos el resultado de vuelta al dominio X, deberíamos obtener exactamente la misma imagen original[6].

Se utilizan dos Generadores y dos Discriminadores simultáneamente:

- **Generadores** G **y** F: Traducen de $X \to Y$ y de $Y \to X$.
- **Discriminador** D_X **y** D_Y: Intenta distinguir entre las imágenes reales de X y las generadas por G (y lo mismo con Y y F).



[14].

[6] Es decir, traducir *Hola*→*Bonjour* y luego *Bonjour*→*Hola*.

Cycle-Consistency Loss

La matemática detrás de este concepto se plasma en la Cycle-Consistency Loss.

Si solo usáramos la pérdida adversaria, el generador podría mapear todos los caballos a una única foto de una cebra genérica que engañara al discriminador (Mode Collapse). Para evitarlo, forzamos a que el proceso sea reversible:

$$\mathcal{L}_{cyc}(G, F) = ||F(G(x)) - x||_1 + ||G(F(y)) - y||_1 \tag{17}$$

- **Forward cycle:** $x \to G(x) \to F(G(x)) \approx x$
- **Backward cycle:** $y \to F(y) \to G(F(y)) \approx y$

Donde básicamente se utiliza la norma L_1 para comparar la calidad del ciclo.

Lineas de investigación de GAN

Image-to-image translation

- CGAN, 2014[10]
- ACGAN, 2016[11]
- pixpix, 2018[15]
- CycleGAN, 2017[15]
- DiscoGAN, 2017[16]
- GauGAN, 2019[17]
- CSGAN, 2019[18]

Mejoras en la calidad

- DCGAN, 2016[19]
- ProGAN, 2017[20]
- StyleGAN, 2018[21]
- Alias-FreeGAN, 2021[22]
- CSGAN, 2021[18]

Lineas de investigación de GAN

Super resolution

- SRGAN, 2017[23]
- srcaGAN, 2021[24]
- WSRGAN, 2021[25]

Crecimiento de las redes

- ProGAN, 2017[20]
- DGGAN, 2021[26]

Referencias i

[1] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio.
Generative adversarial nets.
Advances in neural information processing systems, 27, 2014.

[2] AA Cournot and Augustin Cournot.
Researches into the mathematical principles of the theory of wealth.
1897.

[3] Akash Srivastava, Lazar Valkov, Chris Russell, Michael U Gutmann, and Charles Sutton.
Veegan: Reducing mode collapse in gans using implicit variational learning.
Advances in neural information processing systems, 30, 2017.

Referencias ii

[4] Kevin Shen (Weights and Biases).
Mode collapse image.
https://wandb.ai/authors/DCGAN-ndb-test/reports/Measuring-Mode-Collapse-in-GANs-Using-Weights-Biases--VmlldzoxNzg5MDk, 2022.
[Online; accessed July, 2026].

[5] Tim Salimans, Ian Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen.
Improved techniques for training gans.
Advances in neural information processing systems, 29, 2016.

[6] Martin Arjovsky, Soumith Chintala, and Léon Bottou.
Wasserstein generative adversarial networks.
In *International conference on machine learning*, pages 214–223. Pmlr, 2017.

Referencias iii

[7] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna.
Rethinking the inception architecture for computer vision (2015).
arXiv preprint arXiv:1512.00567, 2015.

[8] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter.
Gans trained by a two time-scale update rule converge to a local nash equilibrium.
Advances in neural information processing systems, 30, 2017.

[9] Mikołaj Bińkowski, Danica J Sutherland, Michael Arbel, and Arthur Gretton.
Demystifying mmd gans.
arXiv preprint arXiv:1801.01401, 2018.

Referencias iv

[10] Mehdi Mirza and Simon Osindero.
Conditional generative adversarial nets.
arXiv preprint arXiv:1411.1784, 2014.

[11] A Odena, C Olah, and J Shlens.
Conditional image synthesis with auxiliary classi er gans arxiv e-prints.(oct.
arXiv preprint stat.ML/1610.09585, 2016.

[12] Yann LeCun.
The mnist database of handwritten digits.
http://yann. lecun. com/exdb/mnist/, 1998.

Referencias v

[13] Ayushman Dash, John Cristian Borges Gamboa, Sheraz Ahmed, Marcus Liwicki, and Muhammad Zeshan Afzal.
Tac-gan-text conditioned auxiliary classifier generative adversarial network.
arXiv preprint arXiv:1703.06412, 2017.

[14] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A Efros.
Unpaired image-to-image translation using cycle-consistent adversarial networks.
In *Proceedings of the IEEE international conference on computer vision*, pages 2223–2232, 2017.

Referencias vi

[15] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A Efros.
Image-to-image translation with conditional adversarial networks.
In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1125–1134, 2017.

[16] Taeksoo Kim, Moonsu Cha, Hyunsoo Kim, Jung Kwon Lee, and Jiwon Kim.
Learning to discover cross-domain relations with generative adversarial networks.
In *International conference on machine learning*, pages 1857–1865. PMLR, 2017.

Referencias vii

[17] Taesung Park, Ming-Yu Liu, Ting-Chun Wang, and Jun-Yan Zhu.
Semantic image synthesis with spatially-adaptive normalization.
In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 2337–2346, 2019.

[18] Kishan Babu Kancharagunta and Shiv Ram Dubey.
Csgan: Cyclic-synthesized generative adversarial networks for image-to-image transformation.
arXiv preprint arXiv:1901.03554, 2019.

[19] Alec Radford, Luke Metz, and Soumith Chintala.
Unsupervised representation learning with deep convolutional generative adversarial networks.
arXiv preprint arXiv:1511.06434, 2015.

Referencias viii

[20] Tero Karras, Timo Aila, Samuli Laine, and Jaakko Lehtinen.
Progressive growing of gans for improved quality, stability, and variation.
arXiv preprint arXiv:1710.10196, 2017.

[21] Tero Karras, Samuli Laine, and Timo Aila.
A style-based generator architecture for generative adversarial networks. arxiv e-prints.
arXiv preprint arXiv:1812.04948, 2018.

[22] Tero Karras, Miika Aittala, Samuli Laine, Erik Härkönen, Janne Hellsten, Jaakko Lehtinen, and Timo Aila.
Alias-free generative adversarial networks.
Advances in Neural Information Processing Systems, 34:852–863, 2021.

Referencias ix

[23] Christian Ledig, Lucas Theis, Ferenc Huszár, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, et al.
Photo-realistic single image super-resolution using a generative adversarial network.
In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4681–4690, 2017.

[24] Baozhong Liu and Ji Chen.
A super resolution algorithm based on attention mechanism and srgan network.
IEEE Access, 9:139138–139145, 2021.

Referencias x

[25] Hangpu Cao and Sicheng Mi.
Weighted srgan and reconstruction loss analysis for accurate image super resolution.
In *Journal of Physics: Conference Series*, volume 1903, page 012050. IOP Publishing, 2021.

[26] Lanlan Liu, Yuting Zhang, Jia Deng, and Stefano Soatto.
Dynamically grown generative adversarial networks.
In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 8680–8687, 2021.

Citación

Cita APA: Iglesias, G. and Talavera, E. (2026, noviembre 7). *Generative Adversarial Networks* [Diapositivas de la asignatura Métodos Generativos]. GitHub. *https://guillermoih.github.io/M-todos-Generativos/Generative_Adversarial_Networks.pdf*

Tema de las diapositivas "Metropolis" lincenciado bajo LPPL 1.3c

BibTeX

```
@slides{Generative_Adversarial_Networks_MG_2026,
   author = {Iglesias, Guillermo and Talavera, Edgar},
   title = {Generative Adversarial Networks [Diapositivas de la asignatura Métodos Generativos]},
   year = {2026},
   month = {11},
   howpublished={\url{
          https://guillermoih.github.io/M-todos-Generativos/Generative_Adversarial_Networks.pdf}},
   note = {[Online; accessed 7 de noviembre de 2026]},
   organization={ETSISI. Universidad Politécnica de Madrid}
}
```